

# Python String Strip Methods

*strip(), lstrip(), rstrip() — with code and corresponding output*

**Key idea:** These methods remove whitespace or selected characters only from the beginning and/or end of a string. They do not remove matching characters from the middle.

## 1. strip() — Remove Whitespace from Both Sides

strip() without an argument removes leading and trailing whitespace from both sides of a string.

### Code

```
text = "  Hello Python  "

print(text.strip())
```

### Output

```
Hello Python
```

## 2. lstrip() — Remove Whitespace from the Left Side

lstrip() removes whitespace only from the beginning (left side) of the string.

### Code

```
text = "  Hello Python  "

print(text.lstrip())
```

### Output

```
Hello Python
```

## 3. rstrip() — Remove Whitespace from the Right Side

rstrip() removes whitespace only from the end (right side) of the string.

### Code

```
text = "  Hello Python  "

print(text.rstrip())
```

### Output

```
Hello Python
```

## 4. strip(chars) — Remove a Specific Character from Both Sides

You can pass one or more characters to strip(). Python will remove those characters repeatedly from both ends.

### Code

```
text = "###Python###"

print(text.strip("#"))
```

### Output

```
Python
```

## 5. lstrip(chars) and rstrip(chars)

The same character-removal behavior can be limited to only the left side or only the right side.

### Code

```
text = "###Python###"

print(text.lstrip("#"))
print(text.rstrip("#"))
```

### Output

```
Python###
###Python
```

## 6. Multiple Characters in strip()

The argument is treated as a set of individual characters, not as one complete word.

### Code

```
text = "aabbccPythonccba"

print(text.strip("abc"))
```

### Output

```
Python
```

**Note:** `strip("abc")` means remove any *a*, *b*, or *c* from both ends until another character is reached.

## 7. strip() Does Not Remove Characters from the Middle

`strip()` only works at the two ends of the string. Matching characters in the middle stay unchanged.

### Code

```
text = "###Python###Programming###"

print(text.strip("#"))
```

### Output

```
Python###Programming
```

## 8. Default Whitespace: Spaces, Tabs and Newlines

Without an argument, `strip()` removes common whitespace such as spaces, tab characters (`\t`), and newline characters (`\n`).

### Code

```
text = "\n\t Python \n\t"

print(text.strip())
```

### Output

```
Python
```

## 9. Practical Example — Cleaning User Input

`strip()` is useful when a user accidentally enters extra spaces before or after a value.

### Code

```
name = input("Enter your name: ")
name = name.strip()

print(f"Hello, {name}!")
```

### Output

```
Enter your name:   Rahul
Hello, Rahul!
```

## 10. Practical Example — Cleaning Lines Read from a File

When reading a text file line by line, each line often ends with a newline character. `strip()` can clean it before use.

### Code

```
with open("fruits.txt") as file:
    for line in file:
        fruit = line.strip()
        print(fruit)
```

### Output

```
Apple
Banana
Mango
```

**Note:** If you only want to remove the newline and preserve other spaces, use `line.rstrip("\n")`.

## 11. Practical Example — Cleaning Comma-Separated Values

After `split(",")`, each item may still have spaces around it. `strip()` is a common way to clean each value.

### Code

```
data = " Python , Java , C++ "

languages = data.split(",")

for language in languages:
    print(language.strip())
```

### Output

```
Python
Java
C++
```

## 12. List Comprehension with `strip()`

You can combine `split()` and `strip()` in one line using a list comprehension.

### Code

```
data = " Python , Java , C++ "
```

```
languages = [x.strip() for x in data.split(",")]
print(languages)
```

#### Output

```
['Python', 'Java', 'C++']
```

### 13. Important Trap — strip() Does Not Remove an Exact Prefix

strip() should not be used when you want to remove a complete prefix such as https://. Its argument is treated as individual characters.

#### Code

```
url = "https://example.com"

print(url.strip("https://"))
```

#### Output

```
example.com
```

**Note:** This example may appear to work, but the logic is unsafe because strip() removes any of the listed characters from both ends.

### 14. Correct Way to Remove an Exact Prefix — removeprefix()

Use removeprefix() when you want to remove one exact prefix from the beginning of a string.

#### Code

```
url = "https://example.com"

print(url.removeprefix("https://"))
```

#### Output

```
example.com
```

### 15. Correct Way to Remove an Exact Suffix — removesuffix()

Use removesuffix() when you want to remove one exact suffix from the end of a string.

#### Code

```
filename = "photo.jpg"

print(filename.removesuffix(".jpg"))
```

#### Output

```
photo
```

### Quick Comparison

| Method           | What it does                           |
|------------------|----------------------------------------|
| text.strip()     | Removes whitespace from both sides     |
| text.lstrip()    | Removes whitespace from the left side  |
| text.rstrip()    | Removes whitespace from the right side |
| text.strip("#")  | Removes # from both sides              |
| text.lstrip("#") | Removes # from the left side           |
| text.rstrip("#") | Removes # from the right side          |

| Method                                | What it does                 |
|---------------------------------------|------------------------------|
| <code>text.removeprefix("abc")</code> | Removes the exact prefix abc |
| <code>text.removesuffix("abc")</code> | Removes the exact suffix abc |

**Memory trick: strip = both sides, lstrip = left, rstrip = right.**